

MacroBase: Static Bug and Reliability Audit

Android offline nutrition tracker (Kotlin, Jetpack Compose, Room, ML Kit)

Project	MacroBase (com.macrobase.app, versionName 1.2.2, versionCode 13)
Audit date	3 October 2026
Version control	Branch master, latest commit ddfc057dabb63fbcc97e1cb7866f7b88ccad2edc ("added rank images"), plus uncommitted changes in 64 tracked files and several untracked files, audited as found in the working tree
Scope	Whole repository: app module (main sources, resources, manifest, bundled database, tests), ppocr-sdk module, build configuration, data-generation tool, documentation
Findings	30 total: P0 0, P1 1, P2 0, P3 19, P4 10

This was a read-only static review. No code was executed or modified.

B. Executive Summary

The audit found **30 issues: P0 0, P1 1, P2 0, P3 19, P4 10**. By confidence: Confirmed 20, Highly likely 6, Potential 4.

Overall assessment: not ready for production use as a calorie tracker. The app's central promise is accurate, private, offline nutrition logging. The built APK requests INTERNET through ML Kit, which contradicts the zero-network invariant.

Most dangerous issues

- **BUG-008 [P1]** The built APK declares INTERNET and ships ML Kit's Google telemetry transport, breaking the zero-network promise

User journeys most at risk: logging catalog foods; editing past entries; restoring a backup; scanning labels from US-format packaging.

The code is generally tidy, uses snapshots for history and has a sizeable test suite (about 290 unit tests). The defects cluster in a few patterns (section G), so a focused fix sequence (section H) removes most of the user impact.

C. Scope, Method, and Coverage

System type: Android mobile app, offline, single user. Kotlin 2.1, Jetpack Compose (Material 3), Navigation Compose, Koin DI, Room 2.6.1 (user database v3), AndroidX DataStore, a read-only bundled SQLite catalog with FTS5, CameraX, Google ML Kit text recognition, and an included PaddleOCR (ONNX Runtime/OpenCV) library module. minSdk 26, targetSdk 35.

Method: reconnaissance of README, AGENTS.md, docs and build files; risk-ordered review of the data layer by the lead auditor; four parallel read-only reviewers (portability and security; scanner; food logging UI; secondary features) whose findings were each re-checked against the source by the lead auditor; a fresh-context adversarial reviewer that tried to disprove the ten most serious findings (none were overturned; two were narrowed and the text adjusted). The bundled database and an older catalog from git history were queried read-only.

Coverage	Areas
Reviewed fully	Room database, entities, DAOs, migrations, DI; built-in database manager and provider; food, diary, recipe, goals, preferences, portability and rank repositories; use cases; domain models; Food Detail, Basket, Search, Custom Foods, Recipes, Home dashboard; backup archive manager, JSON serializer and import/export screen; scanner pipeline (screen, ViewModel, orchestrator, parsers, detectors, normalisation, OCR engines); widget, MainActivity, manifest and backup rules; merged manifests and merger reports
Reviewed partially	Weight, Water, Statistics, Calendar, Preferences and Rank screens (logic and ViewModels; not every composable); design-system components; ppocr-sdk (lifecycle and crash paths only; unused by the app); unit tests (read for coverage, not run); tools/build_indian_food_database.py and the CSV (where they explain bundled data)
Not reviewed	androidTest sources in depth; Compose visual layout and accessibility semantics in detail (no device available); docs beyond what was needed to understand intent
Skipped	build/ and .gradle/ output (except merged manifests and merger reports), .idea/, .kotlin/, binary assets (ONNX models, rank images), vendored Gradle wrapper

About 130 source and test files (roughly 25,000 lines of main Kotlin) plus the bundled database, manifests and build files were reviewed.

D. Priority Table

ID	Priority	Issue	User impact	Confidence	Location
BUG-008	P1	The built APK declares INTERNET and ships ML Kit's Google telemetry transport, breaking the zero-network promise	The app holds network permission and Google's logging transport while the UI says "Never uploaded to the cloud" (ImportExportScreen.kt:305) and AGENTS.md calls zero network non-negotiable.	Confirmed	app/build/intermediates/merged_manifest/release/processReleaseMainManifest/AndroidManifest.xml:26-27, 132-145
BUG-026	P3	Macro-percentage check differs between UI and use case, so some valid goal saves fail silently	Save appears to work; preferences are stored but goals are not, with no message.	Confirmed	domain/model/Goal.kt:56-67
BUG-027	P3	Weight edit dialog rejects its own pre-filled value in comma-decimal locales	"Please enter a valid positive number"; typed "72,5" is also rejected.	Confirmed	feature/weight/WeightScreen.kt:650-656, 724-731, 771-774
BUG-028	P3	Preferences inputs are not validated; comma or cleared values are silently dropped	"Saved" message, but weight unchanged, cleared target reappears, or a negative water goal is stored.	Confirmed	feature/preferences/PreferencesScreen.kt:855-872
BUG-029	P3	Toggling the unit system discards typed preference edits	The typed value vanishes; strategy auto-detection keeps the value from 80, so the two disagree.	Confirmed	feature/preferences/PreferencesScreen.kt:150-182
BUG-030	P3	Tapping the home-screen widget never opens Search and resets the app's back stack	The dashboard opens instead of Search; any screen the user had open is gone.	Confirmed	MainActivity.kt:21-41
BUG-031	P3	A pending fitness-strategy change cannot be cancelled	Tomorrow Cutting is applied anyway.	Confirmed	feature/preferences/PreferencesScreen.kt:189

BUG-032	P3	Rank penalises today before the day is over (up to -20 RR every morning)	Rank drops every morning and recovers during the day.	Confirmed	data/repository/rank/RankRepositoryImpl.kt:165-218
BUG-033	P3	Statistics windows include one extra day and the consistency score is truncated	"96%" instead of 100%; 7-day averages span 8 days.	Confirmed	feature/statistics/StatisticsScreen.kt:144-149, 195-200
BUG-037	P3	Missing fiber, sugar and sodium are stored as 0.0 instead of null in diary snapshots	History shows "0.0 g" where it should show "-"; this 0 is what triggers BUG-010.	Confirmed	data/database/entity/UserEntities.kt:29-31
BUG-038	P3	Food Detail resets the typed quantity and serving on rotation	Quantity returns to the default (or to the stored entry in edit mode).	Highly likely	feature/detail/FoodDetailScreen.kt:428-430
BUG-039	P3	Basket: errors are swallowed and the header shows only the first item's date and meal	Nothing happens on failure; items log to dates the header did not show.	Confirmed	feature/basket/BasketScreen.kt:518-524, 283-296
BUG-040	P3	Dashboard keeps yesterday's date after midnight, so new logs go to the wrong day	Entries land on yesterday; the header changes from "Today" to a date.	Potential	feature/dashboard/HomeScreen.kt:108
BUG-041	P3	Scanner: with a single per-serving column, a second number in the row becomes the per-100 g value	Wrong sodium with only a "Check" badge.	Highly likely	feature/scanner/NutritionLabelParser.kt:1105-1121
BUG-042	P3	Scanner: adjudicator warnings are discarded and critical warnings can be hidden	No visible "calories too high" warning.	Confirmed	feature/scanner/NutritionLabelScanOrchestrator.kt:214-245
BUG-043	P3	Scanner: table-crop OCR coordinates are mixed with full-image coordinates	Intermittently swapped or duplicated nutrient rows.	Highly likely	feature/scanner/NutritionLabelScanOrchestrator.kt:159-175
BUG-044	P3	Scanner: gallery images are decoded at full size on the main thread and can crash with OutOfMemoryError	Freeze, then app crash.	Confirmed	feature/scanner/NutritionLabelScannerScreen.kt:107-122
BUG-045	P3	Scanner: "Calories from Fat" rows take over the fat and calorie values on older US labels	Fat of 72 g/100 g; calories may fall to the footnote "2,000" (then BUG-022).	Highly likely	feature/scanner/NutritionLabelParser.kt:945-953, 1014-1029
BUG-046	P3	Destructive migration fallback wipes all user data on a downgrade or a missed migration	All diary, custom foods, recipes, weight and water history disappear on launch, with no warning.	Potential	core/di/DiModules.kt:91-99
BUG-047	P3	A crafted or very large backup can crash the app; parse errors embed the whole file in the UI	App crash, or a huge error string rendered in a Text (jank/ANR).	Potential	data/portability/BackupArchiveManager.kt:24-28

BUG-048	P4	Widget shows yesterday after midnight; restore refreshes it before the transaction commits	Stale date and intake on the home screen for up to 30+ minutes after midnight.	Highly likely	app/src/main/res/xml/macrobase_widget_info.xml (updatePeriodMillis=1800000)
BUG-049	P4	Calendar and heatmap recolour past days against the current calorie goal	Lowering the goal from 2,000 to 1,600 turns past on-target days red.	Confirmed	feature/calendar/CalendarScreen.kt:152, 180, 200
BUG-050	P4	Values drift when users switch units or re-save unchanged entries	Small unexpected changes after unrelated edits.	Confirmed	feature/preferences/PreferencesScreen.kt:866-872
BUG-051	P4	Merge import treats custom foods as identical when only potassium, calcium or iron differ	An edited mineral value is skipped and counted as "skipped".	Confirmed	data/repository/PortabilityRepositoryImpl.kt:493-506
BUG-052	P4	Project-rule violations: String.format without Locale.US, raw non-ASCII literals, invalid UTF-8 in a scanner regex	Mixed decimal separators; micrograms written with the micro sign are not parsed.	Confirmed	feature/detail/FoodDetailScreen.kt:605, 626, 954, 1010
BUG-053	P4	DataStore read errors are not caught	Crash instead of falling back to defaults on a corrupt preferences file.	Highly likely	data/repository/GoalsRepositoryImpl.kt:222
BUG-054	P4	Search drops accented and non-Latin letters	No results for accented or Hindi queries (the catalog itself is romanised, so impact is limited).	Confirmed	data/provider/LocalFoodDatabaseProvider.kt:287-295 (sanitizeFtsQuery)
BUG-055	P4	Dead code and APK weight: unused PaddleOCR models, stub recent foods, misleading "Gemini" naming	Larger download; an advertised feature (recents) missing.	Confirmed	app/src/main/assets/models (about 30 MB)
BUG-056	P4	Scanner overlays let taps through, and bitmaps may be recycled while ML Kit still reads them	Overlapping scans or a rare native crash.	Potential	feature/scanner/NutritionLabelScannerScreen.kt:422-434
BUG-057	P4	Scanner header detection treats any element containing "100" or "serv" as a header	Lower-quality parses on some labels.	Confirmed	feature/scanner/SpatialTableDetector.kt:397-412

E. Detailed Findings

Findings are ordered by priority, then by number of users affected and severity. Paths are relative to app/src/main/java/com/macrobase/app/ unless they start with app/, tools/ or another top-level folder.

P1 BUG-008 The built APK declares INTERNET and ships ML Kit's Google telemetry transport, breaking the zero-network promise

Confidence: Confirmed (merged manifests and merger reports in app/build) **Category:** Security / Privacy

Area: Build / manifest

Location:

app/build/intermediates/merged_manifest/release/processReleaseMainManifest/AndroidManifest.xml:26-27, 132-145; app/build/outputs/logs/manifest-merger-release-report.txt:607-611; app/build.gradle.kts:97; app/src/main/AndroidManifest.xml

What is wrong: com.google.mlkit:text-recognition pulls in com.google.android.datatransport:transport-backend-cct, which merges INTERNET and ACCESS_NETWORK_STATE plus the TransportBackendDiscovery/JobInfoSchedulerService components (Firelog/CCT usage logging) into the app. The source manifest has no tools:node="remove" overrides.

How it happens: Build any variant; both the debug (8 Sep) and release (26 Aug) merged manifests contain the permissions.

What the user experiences: The app holds network permission and Google's logging transport while the UI says "Never uploaded to the cloud" (ImportExportScreen.kt:305) and AGENTS.md calls zero network non-negotiable.

Impact: Privacy claim is inaccurate; Play Data Safety answers based on it would be wrong; library usage metrics can leave the device.

Root cause: Transitive manifest merge was never audited.

Evidence:

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
...
uses-permission#android.permission.INTERNET
ADDED from [com.google.android.datatransport:transport-backend-cct:2.3.3]
```

Assumptions: The merged manifests are from builds on 26 Aug and 8 Sep; the dependency is unchanged since, so a fresh build should match. Exactly what ML Kit uploads was not verified (no network access in this audit).

Recommended fix: Add xmlns:tools and <uses-permission ... INTERNET tools:node="remove"/> (and ACCESS_NETWORK_STATE) to the app manifest; also remove the datatransport services/receivers, or exclude transport-backend-cct in Gradle. Re-verify OCR still works offline.

Regression test: Build-time check (Gradle task or CI script) that fails if any merged manifest contains android.permission.INTERNET.

Related findings: BUG-018

P3 BUG-026 Macro-percentage check differs between UI and use case, so some valid goal saves fail silently

Confidence: Confirmed **Category:** Bug **Area:** Goals

Location: domain/model/Goal.kt:56-67; domain/usecase/BiometricAndGoalUseCases.kt:139;
feature/preferences/PreferencesScreen.kt:129-133

What is wrong: The UI validates with totalPercentageExact, but UpdateGoalsUseCase uses isValidPercentageSum, which truncates the sum with toInt(). saveAll ignores the failure.

How it happens: Carbs 30.4, protein 34.8, fat 34.8 sums to 99.99999999999999 in floating point; truncation gives 99, the use case fails. A 33.5 x 3 split (100.5) passes.

What the user experiences: Save appears to work; preferences are stored but goals are not, with no message.

Impact: Users believe new targets are active when they are not.

Root cause: Truncating comparison and ignored Result.

Evidence:

```
val totalPercentage: Int
    get() = (carbPercentage + proteinPercentage + fatPercentage).toInt()
val isValidPercentageSum: Boolean
    get() = abs(totalPercentage - 100.0) < 0.01
...
if (goalRes.isSuccess) { onSuccess() }
```

Assumptions: None.

Recommended fix: Validate with `abs(totalPercentageExact - 100.0) < 0.01` and show the failure message.

Regression test: Goal(30.4, 34.8, 34.8).isValidPercentageSum is true; Goal(33.5, 33.5, 33.5) is false.

Related findings: BUG-028

P3 BUG-027 Weight edit dialog rejects its own pre-filled value in comma-decimal locales

Confidence: Confirmed **Category:** Bug / Localization **Area:** Weight screen

Location: feature/weight/WeightScreen.kt:650-656, 724-731, 771-774

What is wrong: Pre-fill uses `String.format("%.1f")` with the default locale ("72,5" in de-DE), and Save parses with `toDoubleOrNull()`.

How it happens: On a German or French device open Edit on a 72.5 kg entry and tap Save without changes.

What the user experiences: "Please enter a valid positive number"; typed "72,5" is also rejected.

Impact: Weight cannot be edited in comma locales.

Root cause: Locale-dependent formatting paired with locale-blind parsing (AGENTS.md 3.1).

Evidence:

```
String.format("%.1f", UnitConversions.kgToLbs(entry.weightKg))
...
val parsed = editInput.toDoubleOrNull()
```

Assumptions: None.

Recommended fix: Format with `Locale.US` and accept ',' when parsing.

Regression test: With `Locale.GERMANY` as default, prefill then parse round-trips 72.5.

Related findings: BUG-012, BUG-052

P3 BUG-028 Preferences inputs are not validated; comma or cleared values are silently dropped

Confidence: Confirmed **Category:** Bug / Validation **Area:** Preferences

Location: feature/preferences/PreferencesScreen.kt:855-872;
data/repository/PreferencesRepositoryImpl.kt:57-61

What is wrong: Weight, height and target weight parse with `toDoubleOrNull()`; null values are skipped on write so the old value survives; water goal falls back to 2,500; negative or zero values are accepted.

How it happens: Type "72,5" for weight, clear target weight, or enter -100 for water and save.

What the user experiences: "Saved" message, but weight unchanged, cleared target reappears, or a negative water goal is stored.

Impact: Profile values drift from what the user entered.

Root cause: No validation; null used as "unchanged".

Evidence:

```
val rawWater = waterGoalInput.toDoubleOrNull() ?: 2500.0
...
preferences.currentWeightKg?.let { prefs[Keys.CURRENT_WEIGHT_KG] = it }
```

Assumptions: None.

Recommended fix: Locale-tolerant parsing, range checks gating Save, and explicit removal when a field is cleared.

Regression test: ViewModel test for clearing a value and rejecting "-5" and "72,5" handling.

Related findings: BUG-027, BUG-029

P3 BUG-029 Toggling the unit system discards typed preference edits

Confidence: Confirmed **Category:** Bug / UX **Area:** Preferences

Location: feature/preferences/PreferencesScreen.kt:150-182

What is wrong: Input fields are `remember(prefs, selectedUnitSystem) { ...from stored prefs... }`, so switching units re-derives them from saved values.

How it happens: Stored weight 75 kg; type 80; toggle to Imperial: the field shows 165.3 lb (75 kg); Save stores 75.

What the user experiences: The typed value vanishes; strategy auto-detection keeps the value from 80, so the two disagree.

Impact: Lost input.

Root cause: State keyed on the unit toggle.

Evidence:

```
var weightInput by remember(prefs, selectedUnitSystem) {  
    val kg = prefs.currentWeightKg
```

Assumptions: None.

Recommended fix: Convert the currently typed text when the unit changes.

Regression test: Compose test: type, toggle, assert converted typed value.

Related findings: BUG-028, BUG-050

P3 BUG-030 Tapping the home-screen widget never opens Search and resets the app's back stack

Confidence: Confirmed **Category:** Bug **Area:** Widget / MainActivity

Location: MainActivity.kt:21-41; feature/widget/MacroBaseWidgetProvider.kt:123-124

What is wrong: The widget intent uses `NEW_TASK | CLEAR_TOP` with standard launch mode, so MainActivity is recreated and `onCreate` runs. `onCreate` calls `tryEmit` on a `MutableSharedFlow` with `replay 0` before `setContent` subscribes; with no subscribers the value is dropped.

How it happens: Tap the widget (app running or not).

What the user experiences: The dashboard opens instead of Search; any screen the user had open is gone.

Impact: The widget's only action does not work.

Root cause: Event emitted before a collector exists.

Evidence:

```
private val widgetActionFlow = MutableSharedFlow<String?>(extraBufferCapacity = 1)  
...  
if (intent?.action == MacroBaseWidgetProvider.ACTION_WIDGET_LOG_FOOD) {  
    widgetActionFlow.tryEmit(intent?.action)  
}
```

Assumptions: None.

Recommended fix: Use a `Channel` or `replay = 1` (consume after handling), or read `intent.action` inside a `LaunchedEffect`; add `FLAG_ACTIVITY_SINGLE_TOP` or `launchMode="singleTop"`.

Regression test: Instrumented test launching with `ACTION_WIDGET_LOG_FOOD` asserts the Search route.

Related findings: BUG-048

P3 BUG-031 A pending fitness-strategy change cannot be cancelled

Confidence: Confirmed **Category:** Bug **Area:** Goals / Preferences

Location: feature/preferences/PreferencesScreen.kt:189; data/repository/GoalsRepositoryImpl.kt:164, 206-213

What is wrong: The selector starts from the active strategy and updateGoals compares against the active strategy, so reverting to it is "no change" and the scheduled keys and tomorrow's transition stay.

How it happens: Maintaining -> Cutting (scheduled for tomorrow) -> back to Maintaining, Save.

What the user experiences: Tomorrow Cutting is applied anyway.

Impact: User intent ignored.

Root cause: Comparison ignores the pending state.

Evidence:

```
var selectedGoal by remember(goals) { mutableStateOf(goals.fitnessGoal) }
```

Assumptions: None.

Recommended fix: Compare against the pending value; clear SCHEDULED_* and tomorrow's record when the selection equals the active strategy.

Regression test: Schedule, revert, advance one day: strategy is still Maintaining.

Related findings: BUG-016

P3 BUG-032 Rank penalises today before the day is over (up to -20 RR every morning)

Confidence: Confirmed (behaviour); design intent unclear **Category:** Bug / UX **Area:** Rank feature

Location: data/repository/rank/RankRepositoryImpl.kt:165-218;
data/repository/CoreRepositoryImplementations.kt:294-295

What is wrong: The evaluation loop includes today, so an unlogged or partly logged day is scored as complete. today is also captured once when the flow is built.

How it happens: Open Rank before breakfast: lifestyle score about 4, delta clamped to -20.

What the user experiences: Rank drops every morning and recovers during the day.

Impact: Confusing, demotivating feedback.

Root cause: Partial day treated as final.

Evidence:

```
var curr = startDate  
while (!curr.isAfter(today)) {
```

Assumptions: None.

Recommended fix: Evaluate through yesterday; show today as a provisional preview not added to the rating.

Regression test: With no entries today, today's delta is excluded from the rating.

Related findings: BUG-023

P3 BUG-033 Statistics windows include one extra day and the consistency score is truncated

Confidence: Confirmed **Category:** Bug **Area:** Statistics

Location: feature/statistics/StatisticsScreen.kt:144-149, 195-200;
data/repository/CoreRepositoryImplementations.kt:294-297

What is wrong: "30 Days" uses today.minusDays(30) inclusive (31 days) and the score uses toInt().

How it happens: Log each of the last 30 days but not day 31.

What the user experiences: "96%" instead of 100%; 7-day averages span 8 days.

Impact: Misreported adherence.

Root cause: Off-by-one range and truncation.

Evidence:

```
val cFrom = today.minusDays(cInt.days)
...
((loggedDays.toDouble() / eligibleCells.size.toDouble()) * 100.0).toInt()
```

Assumptions: None.

Recommended fix: Use minusDays(days - 1) and roundToInt().

Regression test: Exactly 30 logged days gives 30 eligible days and 100%.

Related findings: BUG-034

P3 BUG-037 Missing fiber, sugar and sodium are stored as 0.0 instead of null in diary snapshots

Confidence: Confirmed **Category:** Data integrity / AGENTS.md 2.4 **Area:** Repository + schema

Location: data/database/entity/UserEntities.kt:29-31; data/repository/DiaryRepositoryImpl.kt:108-110, 134-136, 159-161

What is wrong: Columns are NOT NULL with default 0.0 and the mapper writes ?: 0.0, so "unknown" and "zero" become the same value. Saturated fat, trans fat and cholesterol are not snapshotted at all.

How it happens: Log a food with no fiber data.

What the user experiences: History shows "0.0 g" where it should show "-"; this 0 is what triggers BUG-010.

Impact: Misleading micronutrient history.

Root cause: Schema chose non-null defaults.

Evidence:

```
loggedFiber = entry.calculatedNutrition.fiberGrams ?: 0.0,  
loggedSugar = entry.calculatedNutrition.sugarGrams ?: 0.0,  
loggedSodium = entry.calculatedNutrition.sodiumMg ?: 0.0,
```

Assumptions: None.

Recommended fix: Add nullable columns via a non-destructive migration (or a sentinel) and map null through.

Regression test: Log a food with null fiber; reloaded entry has fiberGrams == null.

Related findings: BUG-010, BUG-007

P3 BUG-038 Food Detail resets the typed quantity and serving on rotation

Confidence: Highly likely **Category:** UX **Area:** Food Detail

Location: feature/detail/FoodDetailScreen.kt:428-430

What is wrong: LaunchedEffect(foodId, entryId, basketItemId) runs again when the activity is recreated and loadFood overwrites the retained ViewModel state; the manifest declares no configChanges.

How it happens: Type 2.5, rotate the phone.

What the user experiences: Quantity returns to the default (or to the stored entry in edit mode).

Impact: Lost input; possible wrong log if unnoticed.

Root cause: Unconditional reload.

Evidence:

```
LaunchedEffect(foodId, entryId, basketItemId) {  
    viewModel.loadFood(foodId, mealType, date, entryId, basketItemId)  
}
```

Assumptions: None.

Recommended fix: Skip loadFood when the ViewModel already holds these arguments; use SavedStateHandle.

Regression test: Recreate the activity in a test and assert the typed quantity survives.

Related findings: None

P3 BUG-039 Basket: errors are swallowed and the header shows only the first item's date and meal

Confidence: Confirmed **Category:** UX / Reliability **Area:** Basket

Location: feature/basket/BasketScreen.kt:518-524, 283-296; feature/basket/BasketViewModel.kt:56-70

What is wrong: onError is an empty lambda; the "When:" header mirrors the first item while items keep their own dates/meals; the "Single Food (Recipe)" option is stored but never used.

How it happens: A commit failure, or a basket with mixed dates.

What the user experiences: Nothing happens on failure; items log to dates the header did not show.

Impact: Confusing, silent failures.

Root cause: Unfinished UI wiring.

Evidence:

```
viewModel.submitBasket(  
    onSuccess = onNavigateToDashboard,  
    onError = { /* show error */ }  
)
```

Assumptions: None.

Recommended fix: Show a snackbar on error; show per-item date/meal when they differ; remove or implement the recipe mode.

Regression test: Failing commit shows an error message.

Related findings: BUG-015

P3 BUG-040 Dashboard keeps yesterday's date after midnight, so new logs go to the wrong day

Confidence: Potential **Category:** Bug **Area:** Dashboard

Location: feature/dashboard/HomeScreen.kt:108

What is wrong: _selectedDate is set once when the activity-scoped ViewModel is created and only reset by navigation to Home.

How it happens: Leave the app open overnight, then tap + in the morning.

What the user experiences: Entries land on yesterday; the header changes from "Today" to a date.

Impact: Misfiled meals.

Root cause: No day-rollover handling.

Evidence:

```
private val _selectedDate = MutableStateFlow(LocalDate.now(ZoneId.systemDefault()))
```

Assumptions: The activity is not destroyed overnight (common when the app stays in recents).

Recommended fix: On resume, move the selection to today if it previously was today.

Regression test: Advance a fake clock past midnight; selection follows.

Related findings: BUG-048

P3 **BUG-041 Scanner: with a single per-serving column, a second number in the row becomes the per-100 g value**

Confidence: Highly likely **Category:** Data integrity **Area:** Label scanner parser

Location: feature/scanner/NutritionLabelParser.kt:1105-1121

What is wrong: In the single PER_SERVING branch, any second non-% token is assigned to per100g, which then outranks the per-serving value.

How it happens: US label "Sodium 160mg 7 %" OCR'd as "160mg", "7", "%": per-100 g sodium becomes 7.

What the user experiences: Wrong sodium with only a "Check" badge.

Impact: Wrong micronutrients on US-style labels.

Root cause: Extra token treated as a second column.

Evidence:

```
perServing = valueTokens[0].value
...
if (valueTokens.size >= 2) {
    per100g = valueTokens[1].value
```

Assumptions: OCR splits the % sign into its own element.

Recommended fix: Never fill per100g from extra tokens when only a per-serving column exists; ignore numbers followed by a separate "%".

Regression test: Fixture "Sodium 160mg 7 %" yields per-serving 160.

Related findings: BUG-019

P3 BUG-042 Scanner: adjudicator warnings are discarded and critical warnings can be hidden

Confidence: Confirmed **Category:** UX / Data integrity **Area:** Scanner orchestration + UI

Location: feature/scanner/NutritionLabelScanOrchestrator.kt:214-245;
feature/scanner/NutritionLabelScannerScreen.kt:1190

What is wrong: combinedWarnings is captured before adjudication, so the adjudicator's range and discrepancy warnings never reach the UI; the range check only runs for low-confidence drafts; the modal shows only the first three warnings, with image-quality hints first.

How it happens: A 1,700 kcal/100 g kJ misread (BUG-019) on a slightly blurry photo.

What the user experiences: No visible "calories too high" warning.

Impact: Safety net does not work.

Root cause: Ordering bug and truncated list.

Evidence:

```
val combinedWarnings = quality.warnings + draft.warnings
if (adjudicator.shouldAdjudicate(draft)) { ... draft = adjudicator.adjudicate(draft, tableGrid) }
...
for (warn in draft.warnings.take(3)) {
```

Assumptions: None.

Recommended fix: Build warnings after adjudication, always run range checks, sort critical warnings first or show all.

Regression test: Orchestrator test with fake OCR: a 1,700 kcal/100 g draft has a range warning in the first three.

Related findings: BUG-019

P3 BUG-043 Scanner: table-crop OCR coordinates are mixed with full-image coordinates

Confidence: Highly likely **Category:** Data integrity **Area:** Scanner orchestration

Location: feature/scanner/NutritionLabelScanOrchestrator.kt:159-175; feature/scanner/OcrEnsemble.kt:150-172

What is wrong: The TableCrop pass OCRs a cropped bitmap, but its boxes are never offset back; the ensemble merges lines by Y distance alone.

How it happens: Labels framed with margin above the table.

What the user experiences: Intermittently swapped or duplicated nutrient rows.

Impact: Hard-to-reproduce wrong values.

Root cause: Missing coordinate translation.

Evidence:

```
val cropOcr = ocrEngine.recognizeText(tableCrop)
...
abs(lineY - exY) < 20
```

Assumptions: None.

Recommended fix: Translate crop-pass boxes by the crop origin before scoring and merging.

Regression test: buildEnsemble with an offset crop pass yields one coordinate space and no duplicate rows.

Related findings: BUG-019

P3 BUG-044 Scanner: gallery images are decoded at full size on the main thread and can crash with OutOfMemoryError

Confidence: Confirmed (code); crash depends on image size **Category:** Reliability / Performance **Area:** Scanner screen

Location: feature/scanner/NutritionLabelScannerScreen.kt:107-122

What is wrong: BitmapFactory.decodeStream runs on the main thread without inSampleSize; catch (e: Exception) does not catch OutOfMemoryError. The picked image also bypasses quality checks (isTestImage = true).

How it happens: Pick a 50 MP photo from the gallery.

What the user experiences: Freeze, then app crash.

Impact: Crash on a user-reachable button.

Root cause: Unbounded decode on the UI thread.

Evidence:

```
val bitmap = context.contentResolver.openInputStream(uri)?.use { stream ->
    BitmapFactory.decodeStream(stream)
}
```

Assumptions: The gallery button is visible in production builds (no build-type gate was found).

Recommended fix: Decode off the main thread with bounds probing and sampling; catch Throwable; keep quality checks.

Regression test: Decode a 8000x6000 image under a constrained heap without crashing.

Related findings: None

P3 BUG-045 Scanner: "Calories from Fat" rows take over the fat and calorie values on older US labels

Confidence: Highly likely **Category:** Data integrity **Area:** Label scanner parser

Location: feature/scanner/NutritionLabelParser.kt:945-953, 1014-1029

What is wrong: The "calories from fat" row is excluded only for energy; for fat, the first row matching \bfat\b wins.

How it happens: Row "Calories 230 Calories from Fat 72" above "Total Fat 8g".

What the user experiences: Fat of 72 g/100 g; calories may fall to the footnote "2,000" (then BUG-022).

Impact: Wrong macros on legacy labels.

Root cause: Keyword match not anchored to the row label.

Evidence:

```
if (text.contains("calories from fat") || text.contains("cal from fat")) false
```

Assumptions: None.

Recommended fix: Skip "calories from fat" rows for every nutrient; require the keyword at the start of the row.

Regression test: Legacy US fixture yields fat 8 g and calories 230.

Related findings: BUG-022

P3 BUG-046 Destructive migration fallback wipes all user data on a downgrade or a missed migration

Confidence: Potential **Category:** Data loss / Reliability **Area:** Database setup

Location: core/di/DiModules.kt:91-99; data/database/UserDatabase.kt:19-29

What is wrong: .fallbackToDestructiveMigration() deletes and recreates every table whenever Room finds no migration path, including downgrades. exportSchema = false means there is no schema history to catch a forgotten migration.

How it happens: Install an older APK over a newer one, restore a cloud backup (BUG-018) into an older build, or ship a schema change without a migration.

What the user experiences: All diary, custom foods, recipes, weight and water history disappear on launch, with no warning.

Impact: Catastrophic for the affected user, though the trigger is uncommon.

Root cause: Convenience setting left on for production.

Evidence:

```
.addMigrations(UserDatabase.MIGRATION_1_2, UserDatabase.MIGRATION_2_3)
.fallbackToDestructiveMigration()
```

Assumptions: A downgrade or forgotten migration occurs; current 1->2->3 paths are present.

Recommended fix: Remove the fallback (or limit it to debug), enable exportSchema and add MigrationTestHelper tests for every version.

Regression test: Migration test from v1 and v2 schemas to v3 preserving rows.

Related findings: BUG-018

P3 BUG-047 A crafted or very large backup can crash the app; parse errors embed the whole file in the UI

Confidence: Potential **Category:** Reliability / Security **Area:** Backup / import

Location: data/portability/BackupArchiveManager.kt:24-28; data/portability/BackupJsonSerializer.kt:400-412

What is wrong: Limits allow 100 MB of uncompressed JSON held as byte arrays and strings, then parsed into an object graph; OutOfMemoryError is not caught. Error messages include the entire JSON text.

How it happens: Import a 100 KB zip that expands to about 100 MB, or a large malformed diary.json.

What the user experiences: App crash, or a huge error string rendered in a Text (jank/ANR).

Impact: Crash on import of untrusted files.

Root cause: Generous limits and verbose errors.

Evidence:

```
const val MAX_TOTAL_UNCOMPRESSED_BYTES = 100 * 1024 * 1024L
...
throw IllegalArgumentException("JSON is not an object: $json")
```

Assumptions: A hostile or unusually large archive.

Recommended fix: Lower limits (for example 20 MB), stream-parse, catch OutOfMemoryError, and cap error text to position plus a short snippet.

Regression test: Extraction of a 90 MB-expansion zip fails gracefully; error messages stay under 300 characters.

Related findings: BUG-017

P4 BUG-048 Widget shows yesterday after midnight; restore refreshes it before the transaction commits

Confidence: Highly likely **Category:** Reliability **Area:** Widget

Location: app/src/main/res/xml/macrobse_widget_info.xml (updatePeriodMillis=1800000); data/repository/PortabilityRepositoryImpl.kt:196-238

What is wrong: No receiver for DATE_CHANGED/TIME_SET/TIMEZONE_CHANGED, so the widget refreshes only every 30 minutes or later under Doze. Restore triggers widget updates through updateGoals, which can read pre-commit data.

How it happens: See "What is wrong".

What the user experiences: Stale date and intake on the home screen for up to 30+ minutes after midnight.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
android:updatePeriodMillis="1800000"
```

Assumptions: None.

Recommended fix: Refresh on date/time-change broadcasts or a midnight alarm; refresh after the import transaction.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-030, BUG-040

P4 BUG-049 Calendar and heatmap recolour past days against the current calorie goal

Confidence: Confirmed (behaviour) **Category:** UX / Design **Area:** Calendar

Location: feature/calendar/CalendarScreen.kt:152, 180, 200; data/repository/DiaryRepositoryImpl.kt:394-421

What is wrong: Every day is compared with goals.dailyCalorieGoal instead of the goal in force on that date.

How it happens: See "What is wrong".

What the user experiences: Lowering the goal from 2,000 to 1,600 turns past on-target days red.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
calorieGoal = goals.dailyCalorieGoal,
```

Assumptions: None.

Recommended fix: Use getGoalForDate once BUG-024 is fixed.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-024

P4 BUG-050 Values drift when users switch units or re-save unchanged entries

Confidence: Confirmed **Category:** Data integrity (minor) **Area:** Preferences / Water / Weight

Location: feature/preferences/PreferencesScreen.kt:866-872; feature/water/WaterScreen.kt:549-556; feature/weight/WeightScreen.kt:724-731

What is wrong: Imperial pre-fills round to whole fl oz or one decimal lb and saving converts back, so 2,500 mL becomes 2,513.75 mL and a 250 mL entry becomes 236.6 mL.

How it happens: See "What is wrong".

What the user experiences: Small unexpected changes after unrelated edits.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
UnitConversions.mlToFlOz(entry.amountMl).roundToInt().toString()
```

Assumptions: None.

Recommended fix: Track dirty fields and only write what changed; use finer precision for fl oz.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-029

P4 **BUG-051 Merge import treats custom foods as identical when only potassium, calcium or iron differ**

Confidence: Confirmed **Category:** Data integrity (minor) **Area:** Backup / import

Location: data/repository/PortabilityRepositoryImpl.kt:493-506

What is wrong: isIdenticalTo stops at sodiumMg.

How it happens: See "What is wrong".

What the user experiences: An edited mineral value is skipped and counted as "skipped".

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
sugarGrams == dto.sugarGrams &&  
sodiumMg == dto.sodiumMg
```

Assumptions: None.

Recommended fix: Compare every field.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-006

P4 BUG-052 Project-rule violations: String.format without Locale.US, raw non-ASCII literals, invalid UTF-8 in a scanner regex

Confidence: Confirmed **Category:** Localization / Maintainability **Area:** Many files

Location: feature/detail/FoodDetailScreen.kt:605, 626, 954, 1010; feature/recipes/RecipesScreen.kt:340, 360-362, 481-483, 532; feature/weight/WeightScreen.kt:469, 654, 727, 729, 882; feature/statistics/StatisticsScreen.kt:424, 766-774; feature/scanner/NutritionNumericParser.kt:26; data/portability/BackupArchiveManager.kt:284

What is wrong: AGENTS.md 3.1/3.2 are broken in many places. Most are cosmetic ("12,5 g" in de-DE); WeightScreen ones cause BUG-027. NutritionNumericParser.kt:26 contains raw 0xB5 bytes that are not valid UTF-8, so the "\u00b5g" alternative becomes U+FFFD and never matches.

How it happens: See "What is wrong".

What the user experiences: Mixed decimal separators; micrograms written with the micro sign are not parsed.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
// RecipesScreen.kt:360
Text(text = "P: ${String.format("%.1f", perServing.proteinGrams)}g", ...)
// WeightScreen.kt:727
String.format("%.1f", UnitConversions.kgToLbs(entry.weightKg))
// RecipesScreen.kt:340 contains a raw U+2022 bullet literal between "servings" and the ingredient
count
```

Assumptions: None.

Recommended fix: Pass Locale.US everywhere; replace literals with \u escapes; re-save NutritionNumericParser.kt as UTF-8 with "\u00b5\u03bc".

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-027

P4 BUG-053 DataStore read errors are not caught

Confidence: Highly likely **Category:** Reliability **Area:** Goals / Preferences repositories

Location: data/repository/GoalsRepositoryImpl.kt:222; data/repository/PreferencesRepositoryImpl.kt

What is wrong: No `.catch { IOException }` on DataStore data flows, so a read or corruption error crashes collectors.

How it happens: See "What is wrong".

What the user experiences: Crash instead of falling back to defaults on a corrupt preferences file.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
val prefs = context.goalsDataStore.data.first()
```

Assumptions: None.

Recommended fix: Catch `IOException` and emit `emptyPreferences()`; add a corruption handler.

Regression test: Add a unit or lint check for the specific case described.

Related findings: None

P4 BUG-054 Search drops accented and non-Latin letters

Confidence: Confirmed **Category:** UX **Area:** Food search

Location: data/provider/LocalFoodDatabaseProvider.kt:287-295 (`sanitizeFtsQuery`)

What is wrong: Every character outside `[a-zA-Z0-9 ]` is replaced with a space, so `"jalape\u00f1o"` becomes `"jalape* o*"` and Devanagari input becomes empty, returning no results without trying the LIKE fallback.

How it happens: See "What is wrong".

What the user experiences: No results for accented or Hindi queries (the catalog itself is romanised, so impact is limited).

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
val tokens = raw.replace(Regex("[^a-zA-Z0-9 ]"), " ")
```

Assumptions: None.

Recommended fix: Keep Unicode letters (`\p{L}\p{N}`); the FTS table already uses `unicode61` with `remove_diacritics`.

Regression test: Add a unit or lint check for the specific case described.

Related findings: None

P4 **BUG-055 Dead code and APK weight: unused PaddleOCR models, stub recent foods, misleading "Gemini" naming**

Confidence: Confirmed **Category:** Maintainability / Performance **Area:** Scanner, search

Location: app/src/main/assets/models (about 30 MB); feature/scanner/PaddleOcrEngine.kt:15 (never instantiated); data/repository/FoodRepositoryImpl.kt:78-80;
feature/scanner/NutritionLabelGeminiAdjudicator.kt

What is wrong: About 30 MB of ONNX models plus onnxruntime/OpenCV ship but PaddleOcrEngine is never created. getRecentFoods returns an empty list, so "Recently logged" never appears. The Gemini-named adjudicator contains no network code but has unused apiKey parameters, which invites someone to add a cloud call.

How it happens: See "What is wrong".

What the user experiences: Larger download; an advertised feature (recents) missing.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
override suspend fun getRecentFoods(limit: Int): List<Food> {  
    return emptyList()  
}
```

Assumptions: None.

Recommended fix: Remove unused models/libraries or wire them in; implement recents from diary_entries; rename the adjudicator and drop apiKey.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-008

P4 BUG-056 Scanner overlays let taps through, and bitmaps may be recycled while ML Kit still reads them

Confidence: Potential **Category:** Reliability **Area:** Scanner screen / orchestrator

Location: feature/scanner/NutritionLabelScannerScreen.kt:422-434;
feature/scanner/NutritionLabelScanOrchestrator.kt:125-131

What is wrong: The analyzing overlay is a plain Box with no pointer handling, so the gallery/back/torch buttons stay tappable; on cancellation the finally block recycles variant bitmaps that an in-flight recognizer task may reference.

How it happens: See "What is wrong".

What the user experiences: Overlapping scans or a rare native crash.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
for (v in variants) {  
    if (v.bitmap != safeBitmap && v.bitmap != bitmap) {  
        v.recycleIfNeeded()  
    }  
}
```

Assumptions: None.

Recommended fix: Consume pointer input on overlays; recycle only after ML Kit tasks finish.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-044

P4 BUG-057 Scanner header detection treats any element containing "100" or "serv" as a header

Confidence: Confirmed **Category:** Data integrity (minor) **Area:** Label scanner

Location: feature/scanner/SpatialTableDetector.kt:397-412

What is wrong: Values such as "1004kJ" or "100mg" push headerBottomY down, so rows above are skipped by the spatial path and fall back to the weaker legacy parser.

How it happens: See "What is wrong".

What the user experiences: Lower-quality parses on some labels.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
elements.filter { it.text.contains("100") || it.text.contains("serv", ignoreCase = true) }
```

Assumptions: None.

Recommended fix: Only use elements already classified as column headers.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-019

F. User Journey Failures

First install and catalog setup. The built APK requests INTERNET via ML Kit (BUG-008).

Search and log a catalog food (main workflow). Search drops accented and non-Latin letters (BUG-054). Rotation resets input (BUG-038). The basket editor swallows errors (BUG-039). Overnight sessions log to yesterday (BUG-040).

Edit or delete past diary entries. Missing fiber, sugar and sodium are stored as 0.0 instead of null in diary snapshots (BUG-037).

Nutrition label scanner. Column confusion (BUG-041, BUG-045, BUG-057), coordinate mixing (BUG-043), warnings lost (BUG-042), gallery OOM crash (BUG-044), overlay touch-through (BUG-056).

Backup, restore and device migration. Restore can crash on large archives (BUG-047), and Merge ignores mineral changes (BUG-051). A downgrade wipes data (BUG-046).

Goals, preferences, weight and water. Goal saves can fail silently (BUG-026); a pending strategy cannot be cancelled (BUG-031); preference inputs are dropped or unvalidated (BUG-028, BUG-029); weight editing fails in comma locales (BUG-027); units drift (BUG-050); DataStore errors crash (BUG-053).

Statistics, calendar, rank and widget. Rank penalises the current day (BUG-032). Statistics windows are off by one (BUG-033); the calendar recolours history (BUG-049). The widget's tap action is dropped (BUG-030) and it goes stale at midnight (BUG-048).

G. Root-Cause Patterns

Unknown values treated as a convenient default. Missing micronutrients become 0.0 (BUG-037). This turns "unknown" into a confident wrong number.

Silent failure and success-by-default UI. Results are ignored or errors swallowed (BUG-026, BUG-039, BUG-042), so the user sees success when nothing or the wrong thing was saved.

Locale-blind number handling. String.format without Locale.US paired with toDoubleOrNull parsing produces failures in comma locales (BUG-027, BUG-028, BUG-052).

Tests that do not touch the real implementations. Portability tests use a fake repository, and the merged manifest is never checked. The suite (about 290 tests) passes while the defects above ship.

H. Recommended Fix Order

1. Protect history. BUG-037 with a non-destructive migration; BUG-046 (remove the destructive fallback before the next schema change).

2. Make backups trustworthy. BUG-047 and BUG-051.

3. Restore the privacy promise. BUG-008 (tools:node="remove" plus a CI check).

4. Scanner accuracy. BUG-042, then BUG-041, BUG-043, BUG-045, BUG-044.

5. Basket completeness. BUG-039.

6. Goals, rank, statistics, widget. BUG-026, BUG-031, BUG-032, BUG-033, BUG-030, then the remaining P3/P4 items.

I. Test Plan

- Edit-mode tests for a FoodDetailViewModel with null secondary nutrients. Loaded and saved values must equal the snapshot unless the user changes quantity, and then scale by $\text{newQty} / \text{oldQty}$. (BUG-037)

- Real-repository portability round trip (Room in-memory + fake DataStore): export, archive, validate, Overwrite and Merge import; assert every column. Negative case: oversized archive. (BUG-047, BUG-051)
- Input parsing tests under Locale.GERMANY for weight and preferences. (BUG-027, BUG-028)
- Build check: fail CI if any merged manifest contains android.permission.INTERNET. (BUG-008)
- Scanner golden fixtures: "Sodium 160mg 7 %", legacy "Calories from Fat"; plus an orchestrator test asserting warnings survive adjudication. (BUG-041, BUG-042, BUG-045)
- Migration tests with MigrationTestHelper from v1 and v2 (requires exportSchema = true). (BUG-046)

J. Remaining Uncertainty

- Nothing was executed. No build, unit test, instrumented test or app run took place, so every finding is based on reading and tracing code. "Confirmed" means confirmed by code tracing.
- The audit covered the working tree as found: branch master at ddfc057 plus 64 uncommitted modified files and several untracked files (rank, widget, scanner additions). Results may differ from any released build.
- BUG-008 relies on merged manifests from earlier builds (26 Aug release, 8 Sep debug). The dependency is unchanged, so a new build should match, but this was not rebuilt. What ML Kit's transport actually uploads was not observed (no network access).
- Device-specific behaviour (OCR element splitting, image sizes) affects BUG-041, BUG-043 and BUG-044; these are marked Highly likely or Potential accordingly.
- The ppocr-sdk module was reviewed only for lifecycle and crash risks; it is not instantiated by the app (BUG-055). The Python data tool and the CSV were read only where they explain the bundled data. androidTest sources were not reviewed in depth.
- No file in the repository contained instructions aimed at an AI auditor that looked malicious; NutritionLabelGeminiAdjudicator.kt contains no network code despite its name.

Priority levels with zero findings: P0, P2.